

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №3
з дисципліни: «Поглиблене вивчення JavaScript»
з теми: «Розробка менеджера задач із глобальним станом»

Виконав:
ст. гр. ПЗПІ-23-2
Малишкін Андрій

Перевірив:
посада викладача (уточнити)
ПІБ викладача (уточнити)

3 РОЗРОБКА МЕНЕДЖЕРА ЗАДАЧ ІЗ ГЛОБАЛЬНИМ СТАНОМ

3.1 Мета роботи

Метою роботи є формування практичних навичок організації глобального стану у Vue-застосунках за допомогою бібліотеки Pinia, опанування маршрутизації через Vue Router, розуміння принципів масштабування компонентної архітектури та побудови багатосторінкового SPA з централізованим керуванням даними.

3.2 Завдання

У межах лабораторної роботи необхідно:

- встановити та підключити бібліотеки Pinia і Vue Router;
- налаштувати маршрутизацію з п'ятьма основними маршрутами;
- створити Pinia-сховище з «state», «getters» та «actions» для керування задачами;
- реалізувати сторінку списку задач з фільтрацією за категорією, пріоритетом та пошуком за назвою;
- реалізувати сторінку деталей задачі з можливістю видалення;
- реалізувати форму створення та редагування задачі з валідацією;
- реалізувати сторінку статистики на основі getters сховища;
- забезпечити узгодженість стану між усіма сторінками;
- реалізувати збереження стану у «localStorage», анімації переходів між маршрутами та прогрес-бар виконання задач.

3.3 Хід роботи

3.3.1 Теоретичні відомості

Було розглянуто проблему prop drilling — передачі даних через проміжні компоненти, які самі ці дані не використовують. Для усунення цієї проблеми застосовано глобальне сховище стану Pinia, яке складається з трьох частин: «state» (реактивні дані), «getters» (обчислювані властивості) та «actions» (методи зміни стану).

Для організації багатосторінкового SPA використано Vue Router. Маршрут визначає відповідність між URL і компонентом. Динамічні параметри (наприклад,

«/tasks/:id») доступні через хук «useRoute», а програмні переходи — через «useRouter». У кореновому компоненті розміщено «RouterView» для відображення поточної сторінки та навігаційну панель з «RouterLink».

Окремо було опрацьовано валідацію форм (обов'язковість назви, перевірка дедлайну) та збереження стану між сесіями через серіалізацію масиву задач у «localStorage».

3.3.2 Етапи реалізації застосунку

Під час практичної реалізації було послідовно виконано такі етапи:

- підготовлено проєктну структуру Vue 3 + TypeScript у директорії «src»;
- встановлено та підключено «pinia» і «vue-router» у «main.ts»;
- створено конфігурацію маршрутів у «router/index.ts»;
- реалізовано Pinia-сховище «stores/taskStore.ts» з CRUD-операціями та getters для статистики;
- реалізовано views: список задач, деталі, форма створення/редагування, статистика, сторінку 404;
- додано фільтрацію, пошук, сортування та візуальне виділення прострочених задач;
- налаштовано збереження стану у «localStorage» через «watch»;
- додано анімації переходів між маршрутами через компонент «Transition»;
- оформлено інтерфейс у темній темі з використанням Tailwind CSS та shadcn-подібних UI-компонентів.

3.3.3 Опис архітектури та компонентів

Застосовано багатошарову структуру:

- «App.vue»: навігаційна панель та «RouterView» з анімацією переходів;
- «router/index.ts»: конфігурація маршрутів SPA;
- «stores/taskStore.ts»: глобальний стан задач, getters та actions;
- «views/TaskListView.vue»: список з локальними фільтрами та «computed» для відображення;
- «views/TaskDetailView.vue»: деталі задачі за параметром маршруту;

- «views/TaskFormView.vue»: спільна форма створення та редагування з валідацією;
- «views/StatsView.vue»: статистика виключно на основі getters сховища;
- «components/TaskListToolbar.vue», «TaskListItem.vue», «AppNav.vue»: UI-компоненти;
- «UIButton.vue», «UiInput.vue», «UiCard.vue»: базові UI-примітиви.

Такий поділ дозволив відокремити глобальну бізнес-логіку від представлення та спростити навігацію між сторінками.

3.3.4 Короткі фрагменти програмного коду

Нижче наведено ключові фрагменти реалізації, що демонструють основні принципи роботи застосунку.

3.3.4.1 Pinia-сховище задач

```
export const useTaskStore = defineStore('tasks', {
  state: () => ({
    tasks: loadTasksFromStorage() ?? createDefaultTasks(),
  }),
  getters: {
    overdueTasks(state): Task[] {
      return state.tasks.filter((task) => isOverdue(task))
    },
    statsSummary(state) {
      const total = state.tasks.length
      const completed = state.tasks.filter((t) => t.completed).length
      return {
        total,
        completed,
        completionRate: total === 0 ? 0 : Math.round((completed /
total) * 100),
      }
    },
  },
  actions: {
    addTask(payload: TaskInput): Task {
      const task = { id: Date.now(), ...payload, completed: false }
      this.tasks.unshift(task)
      return task
    },
  },
})
```

3.3.4.2 Конфігурація маршрутів

```
const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', component: TaskListView },
    { path: '/tasks/new', component: TaskFormView },
    { path: '/tasks/:id/edit', component: TaskFormView },
    { path: '/tasks/:id', component: TaskDetailView },
    { path: '/stats', component: StatsView },
  ],
})
```

3.3.4.3 Валідація форми задачі

```
function validateForm(): boolean {
  resetErrors()
  let isValid = true

  if (!form.title.trim()) {
    errors.title = "Назва є обов'язковим полем."
    isValid = false
  }

  if (!isEditMode.value && form.deadline < todayIsoDate()) {
    errors.deadline = 'Дедлайн не може бути у минулому.'
    isValid = false
  }

  return isValid
}
```

3.4 Результати роботи

У результаті виконання лабораторної роботи отримано працездатний менеджер задач на Vue 3. Було забезпечено CRUD-операції, маршрутизацію між п'ятьма сторінками, фільтрацію та сортування списку, сторінку статистики на основі getters, валідацію форм, збереження стану у «localStorage», анімації переходів та прогрес-бар виконання.

Під час перевірки працездатності було підтверджено:

- коректну роботу всіх маршрутів;
- узгодженість даних між списком, деталями, формою та статистикою;
- збереження задач після перезавантаження сторінки;
- відсутність помилок lint/typescheck у проєкті.

3.5 Висновки

Під час виконання лабораторної роботи було закріплено практичні підходи до організації глобального стану та маршрутизації у Vue 3. Було відпрацьовано застосування Pinia («state», «getters», «actions»), Vue Router («RouterView», «RouterLink», «useRoute», «useRouter»), компонентного підходу та розділення локального і глобального стану, а також отримано навички побудови масштабованого багатосторінкового SPA.

3.6 Контрольні запитання

а) Що таке prop drilling і яку проблему він спричиняє?

Prop drilling — передача props через проміжні компоненти, які самі ці дані не використовують. Це ускладнює підтримку коду та робить рефакторинг крихким.

б) З яких трьох частин складається Pinia-сховище? Яке призначення кожної?
«state» — реактивні дані; «getters» — обчислювані властивості на основі state; «actions» — методи для зміни state та асинхронних операцій.

в) Як компонент отримує доступ до даних Pinia-сховища?

Через виклик функції «useTaskStore()» (або іншого store) у setup-компоненті після підключення Pinia до застосунку.

г) Що таке маршрут у Vue Router? Як визначаються динамічні параметри маршруту?

Маршрут — відповідність між URL і компонентом. Динамічні параметри записуються через двокрапку у шляху, наприклад «/tasks/:id».

д) Яке призначення компонентів RouterView та RouterLink?

«RouterView» — місце відображення компонента поточного маршруту; «RouterLink» — декларативне посилання для переходу між маршрутами без перезавантаження сторінки.

е) Як отримати параметри поточного маршруту у компоненті?

Через хук «useRoute()» і властивість «route.params», наприклад «route.params.id».

ж) Як програмно перенаправити користувача на інший маршрут?

Через хук «useRouter()» і метод «router.push('/path')» або «router.replace('/path')».

и) Чим відрізняються getters від actions у Pinia-сховищі?

Getters обчислюють похідні дані з state (як «computed»); actions змінюють state або виконують операції з побічними ефектами.

к) Як реалізувати збереження стану сховища між перезавантаженнями сторінки?

Серіалізувати state у «localStorage» при зміні (наприклад, через «watch») і зчитувати при ініціалізації store або за допомогою плагіна «pinia-plugin-persistedstate».

л) Яку перевагу дає винесення логіки фільтрації у getters сховища порівняно з реалізацією у кожному компоненті окремо?

Уникається дублювання логіки, забезпечується єдине джерело правди для похідних даних (наприклад, статистики) і спрощується підтримка коду.